

Introduction to Terraform

Infrastructure as Code for IT Students

Minh Huynh
Solution Architect - TechX

Course Agenda

01

IaC & Terraform Overview

03

Terraform Workflow

05

Modules & Code Reuse

07

Real-World Project & Next Steps

02

HCL Syntax & Core Concepts

04

State Management

06

Best Practices & Teamwork

MODULE 01

IaC & Terraform Overview

What is Infrastructure as Code (IaC)?

Manual (Old Way)

Traditional Approach

- Manually configure servers via GUI/SSH
- Steps are undocumented & hard to repeat
- Human errors are common
- No version history

IaC (Modern Way)

IaC Approach

- Describe infrastructure in code files
- Version-controlled (Git)
- Repeatable & automated
- Self-documenting

Why Do We Need IaC?

Consistency

Same config every deploy — no more "works on my machine" infrastructure.

Speed

Spin up full environments in minutes, not days.

Version Control

Track every change in Git.
Rollback anytime.

Collaboration

Teams review infra changes via Pull Requests — just like code.

Terraform vs Other Tools

Terraform

Terraform

- Cloud-agnostic (AWS, Azure, GCP...)
- Declarative HCL language
- State-based planning
- Large provider ecosystem
- Best for: multi-cloud infra provisioning

Other Tools

Competitors

- Ansible — config management, agentless
- CloudFormation — AWS-only, JSON/YAML
- Pulumi — uses real languages (Python/JS)
- CDK — AWS-specific, code-first

How Terraform Works — Architecture Overview

- You write .tf configuration files describing desired state
- Terraform Core reads configs & queries state
- Providers translate resources into API calls
- State file tracks what Terraform actually deployed
 - Provider examples: AWS, Azure, GCP, Kubernetes, GitHub, Datadog...

Lab: Install Terraform & Set Up Environment

- 1. Install Terraform CLI: <https://developer.hashicorp.com/terraform/install>
- 2. Verify: `terraform -version`
- 3. Install VS Code Extension: HashiCorp Terraform
- 4. Configure AWS CLI credentials (`aws configure`)
- 5. Explore: registry.terraform.io — browse providers & modules
 - Tip: use `tfenv` to manage multiple Terraform versions

MODULE 02

HCL Syntax & Core Concepts

HCL — HashiCorp Configuration Language

- Declarative: you describe WHAT, not HOW
- Human-readable with good editor support
- File extension: .tf (or .tf.json for JSON syntax)
- Block types: resource, variable, output, locals, data, provider, module, terraform
 - HCL is NOT a general-purpose programming language — no loops like Python

HCL Data Types

Primitive

Primitive Types

- string → "hello"
- number → 42 or 3.14
- bool → true / false

Complex

Complex Types

- list → ["a", "b", "c"]
- map → { key = "value" }
- object → { name = string, port = number }
- tuple → [string, number, bool]

Declaring a Resource Block

```
# Basic resource block structure
resource "aws_instance" "web_server" {
  ami          = "ami-0c02fb55956c7d316"
  instance_type = var.instance_type
  tags = {
    Name          = "WebServer"
    Environment = var.environment
  }
}

# Variable declaration
variable "instance_type" {
  type        = string
  default     = "t2.micro"
  description = "EC2 instance type"
}
```



<resource_type> <local_name> — always two labels

Variables, Outputs & Locals

```
# Input variable
variable "region" {
    type    = string
    default = "us-east-1"
}

# Local value (computed internally)
locals {
    name_prefix = "${var.environment}-${var.project}"
}

# Output value (exported for other modules)
output "instance_ip" {
    value          = aws_instance.web_server.public_ip
    description    = "Public IP of the web server"
}
```

💡 *Locals avoid repetition; outputs expose values to callers*

Data Sources — Read Existing Infrastructure

```
# Fetch the latest Amazon Linux 2 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

# Reference in a resource
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
}
```



Data sources READ existing resources — they do not create anything

MODULE 03

Terraform Workflow

The 4 Core Terraform Commands

terraform init

Downloads providers & modules.
Initializes the backend.
Run once per project.

terraform plan

Shows a diff of what will change.
Does NOT modify real resources.
Always review before apply.

terraform apply

Applies the planned changes.
Prompts for confirmation.
Updates state file.

terraform destroy

Destroys ALL managed resources.
Use with caution!
Great for lab cleanup.

Reading terraform plan Output

```
$ terraform plan
```

Terraform will perform the following actions:

```
# aws_instance.web_server will be created
+ resource "aws_instance" "web_server" {
  + ami           = "ami-0c02fb55956c7d316"
  + instance_type = "t2.micro"
  + tags          = {
    + "Name" = "WebServer"
  }
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Providers — Connecting to Cloud APIs

- Providers are plugins that know how to talk to specific APIs
- Declared in terraform {} block with required_providers
- Versioned independently from Terraform CLI
- terraform init downloads providers from registry.terraform.io
 - Popular providers: hashicorp/aws, hashicorp/azurerm, hashicorp/google, hashicorp/kubernetes

Provider Configuration

```
# Provider version pinning (best practice)
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"    # allows 5.x but not 6.x
    }
  }
}

# Configure the AWS provider
provider "aws" {
  region  = "ap-southeast-1"    # Singapore – closest to Vietnam
  profile = "default"
}
```

🔒 *Pin provider versions to avoid breaking changes in CI/CD pipelines*

Lab: Deploy an EC2 Instance on AWS (Free Tier)

- 1. Create a new directory: `mkdir terraform-lab && cd terraform-lab`
- 2. Write `main.tf` with `provider + aws_instance` resource
- 3. Run `terraform init` → `terraform plan` → `terraform apply`
- 4. Verify in AWS Console: EC2 dashboard
- 5. Clean up: `terraform destroy`
 - Use `t2.micro` or `t3.micro` — included in AWS Free Tier (750h/month)

MODULE 04

State Management

What is Terraform State?

- State = Terraform's memory of what it has deployed
- Stored in terraform.tfstate (JSON format)
- Maps your config resources → real cloud resource IDs
- Without state, Terraform cannot know what exists or what changed
 - Never manually edit terraform.tfstate — always use terraform state commands
 - terraform.tfstate may contain sensitive data (passwords, keys) — protect it!

Remote State — S3 + DynamoDB

```
# Store state remotely (team-friendly)
terraform {
  backend "s3" {
    bucket      = "my-company-terraform-state"
    key         = "prod/vpc/terraform.tfstate"
    region     = "ap-southeast-1"
    encrypt     = true

    # DynamoDB table for state locking
    dynamodb_table = "terraform-state-lock"
  }
}
```

 Remote state prevents two people applying at the same time (state locking)

Terraform State Commands

- `terraform state list` — list all managed resources
- `terraform state show <addr>` — show details of one resource
- `terraform state mv <old> <new>`— rename/move resource in state
- `terraform state rm <addr>` — remove resource from state (does NOT delete cloud resource)
- `terraform import <addr> <id>` — bring existing resource under Terraform management
 - Example: `terraform import aws_s3_bucket.data my-existing-bucket-name`

Common State Mistakes & How to Avoid Them

Common Mistakes

- Committing tfstate to Git (exposes secrets)
- Two teammates running apply simultaneously
- Manually deleting cloud resources → state drift
- Losing the state file (no backup)

Solutions

- Use .gitignore: *.tfstate, *.tfstate.backup
- Use remote state with DynamoDB locking
- Run terraform plan to detect drift; use terraform refresh
- Enable S3 versioning on your state bucket

MODULE 05

Modules & Code Reuse

Why Use Modules? — DRY Principle in IaC

- DRY = Don't Repeat Yourself
- Without modules: copy-paste the same VPC config for dev, staging, prod
- With modules: write once, call three times with different variables
- Modules = reusable, testable, versioned infrastructure components
 - Think of modules like functions in programming

Standard Module Structure

- my-module/
 - main.tf — resource definitions
 - variables.tf — input variables
 - outputs.tf — output values
 - README.md — documentation (required for public modules)
- Calling a module: `module "vpc" { source = "../modules/vpc" ... }`
- Public modules: registry.terraform.io/modules

Calling a Module — Example

```
# -- Calling local module -----  
module "vpc" {  
    source = "../modules/vpc"  
  
    vpc_cidr          = "10.0.0.0/16"  
    public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]  
    environment       = "production"  
}  
  
# -- Using module output -----  
resource "aws_instance" "app" {  
    ami          = data.aws_ami.amazon_linux.id  
    instance_type = "t3.small"  
    subnet_id    = module.vpc.public_subnet_ids[0]  
}
```

 *Module outputs are referenced as `module.<name>.<output_name>`*

Real-World Project Structure (Dev/Staging/Prod)

- infra/
 - modules/vpc/ ← reusable VPC module
 - modules/ec2/ ← reusable EC2 module
 - environments/dev/main.tf ← dev config
 - environments/staging/main.tf ← staging config
 - environments/prod/main.tf ← prod config
- Each environment calls the same modules with different variable values

MODULE 06

Best Practices & Teamwork

Managing Secrets Safely

Don't Do This

- Never hardcode access keys in .tf files
- Never commit *.tfstate or *.tfvars with secrets to Git
- Never put passwords in variable default values

Do This Instead

- Use environment variables: `AWS_ACCESS_KEY_ID`
- Use AWS IAM Roles (EC2/ECS/Lambda) instead of keys
- Use AWS Secrets Manager or HashiCorp Vault
- Encrypted remote state (S3 with SSE-KMS)

Terraform with Git — .gitignore & Naming Conventions

- .gitignore essentials: .terraform/ *.tfstate *.tfstate.* *.tfvars crash.log
- Naming pattern: <env>-<resource>-<purpose> e.g. "prod-rds-main"
- Always use tags: Environment, Project, Owner, ManagedBy = Terraform
- Use terraform fmt before every commit — keeps formatting consistent
- terraform validate — catches syntax errors early
 - tflint — linter for Terraform best practices (install separately)

Using tfvars for Environments

```
# variables.tf
variable "environment" { type = string }
variable "instance_type" { type = string }
variable "db_password" {
  type      = string
  sensitive = true # hides value from plan output
}
```

```
# dev.tfvars
environment = "development"
instance_type = "t2.micro"
```

```
# prod.tfvars
environment = "production"
instance_type = "t3.medium"
```

💡 Apply with specific tfvars: *Supply .tfvars file with non-sensitive values; keep secrets in environment variables*

```
# terraform apply -var-file="prod.tfvars"
```

Terraform in CI/CD Pipelines

- Goal: automate plan & apply through Pull Requests
- GitHub Actions / GitLab CI: run terraform plan on every PR
- Terraform Cloud: managed remote runs, policy checks, team access
- Atlantis: open-source PR automation for Terraform
 - Workflow: PR opened → plan runs → team reviews → merge → apply runs automatically

MODULE 07

Real-World Project & Next Steps

Final Project: Deploy a Web App on AWS

- Architecture: VPC + Public/Private Subnets + EC2 + RDS + S3
- Step 1: Create VPC module with public & private subnets
- Step 2: Deploy EC2 instance in public subnet (web server)
- Step 3: Deploy RDS MySQL in private subnet
- Step 4: Create S3 bucket for static assets
- Step 5: Configure security groups to allow only required traffic
 - All state stored in S3 backend with DynamoDB locking

Debugging Common Errors

Error Type

- Dependency cycle
- Provider version conflict
- State drift (manual changes)
- Resource already exists
- Permission denied

How to Fix

- Use explicit `depends_on` or restructure references
- Pin versions in `required_providers`
- `terraform plan` detects drift; `terraform import` or `refresh`
- Import existing resource or use data source
- Check IAM role/policy attached to your credentials

Advanced Topics to Explore Next

Terraform + Kubernetes

hashicorp/kubernetes provider — manage K8s resources with Terraform.

Terragrunt

Wrapper that adds DRY configs and remote state management.

OpenTofu

Open-source fork of Terraform — community-driven, fully compatible.

HashiCorp Certification

Terraform Associate — entry-level exam. Great for DevOps job search.

Learning Resources & Community

-  Official Docs: developer.hashicorp.com/terraform/docs
-  Provider Registry: registry.terraform.io
-  HashiCorp Learn (free tutorials): developer.hashicorp.com/terraform/tutorials
-  More lab: <https://devopsvn.tech/terraform-series/terraform>
-  Discuss Forum: discuss.hashicorp.com
 - Recommended certification path: Terraform Associate → AWS SAA → AWS DevOps Pro

You're Ready to Infrastructure as Code!

- ✓ IaC fundamentals & Terraform architecture
- ✓ HCL syntax, variables, outputs, data sources
- ✓ Full workflow: init → plan → apply → destroy
- ✓ State management & remote backend
- ✓ Modules, best practices & CI/CD integration